

Guide to make the report

SAEISS ADDC – Technical Report Submission Guidelines

Organized by SAEINDIA Southern Section (SAEISS)

1. Overview

All participating teams of the SAEISS Autonomous Drone Development Challenge (ADDC) must submit a concise Technical Report detailing their design, analysis, fabrication, and testing of the drone. The report should demonstrate engineering understanding, innovation, and validation through testing.

2. Format and Submission

File Format	PDF (.pdf)
File Name	TeamName_TeamID_Report.pdf
Font & Size	Times New Roman, 12 pt, 1.15 line spacing
Page Size	A4 with 1-inch margins
Page Limit	Maximum 15 pages including figures and tables
Submission	Through the official portal or as directed by organizers

3. Recommended Report Structure

1. Team and Project Information
2. Summary and Mission objectives
3. Mission Concept & System Architecture
4. Design Overview
5. Autonomous Navigation Strategy
6. Computer Vision & QR Detection
7. Structural / Aerodynamic / Other analysis
8. Safety & Risk Assessment
9. Unique Technological Development
10. Conclusion and Appendix – ~~2D~~ drafting sheet has to be attached
11. References

Summary and Mission Objectives

USE BUZZZZWORDS

Mission Concept & System Architecture

Mission Planner Control: Automated Polygon-Based Grid Search and Dynamic Guided Target Intervention System

Search and Rescue (SAR), aerial survey, and tactical reconnaissance missions demand high spatial coverage, repeatability, and immediate response upon target acquisition. Manual flight path planning or arbitrary area traversal often leads to search gaps, uneven sensor overlap, and increased operator workload.

This details an end-to-end architecture of an automated search system using ArduPilot and Mission Planner. The operational framework leverages Mission Planner's polygon-based lawnmower (grid) survey generator, establishes reliable telemetry and companion computing interfaces, and executes automated transitions from AUTO mission execution to GUIDED intervention mode upon real-time target detection.

1. Operational Utility of Automated Lawnmower (Grid) Search Patterns

A. Systematic Spatial Coverage

The automatically generated grid (lawnmower) pattern transforms a user-defined complex geographic boundary into a continuous, non-overlapping flight path consisting of parallel flight lines and calculated turnarounds.

- **Elimination of Search Gaps:** Manual navigation risks missing terrain due to perspective distortion and drift. The grid algorithm guarantees that every point within the defined polygon falls within the Sensor Field of View (FOV).
- **Controlled Overlap:** By factoring in the camera focal length, sensor dimensions, flight altitude, and target Ground Sample Distance (GSD), Mission Planner calculates precise front-lap (e.g., 75%) and side-lap (e.g., 60%) percentages. This ensures continuous optical coverage regardless of vehicle speed or wind vector.
- **Flight and Power Efficiency:** The generator aligns flight paths with prevailing wind vectors or the longest polygon axis, minimizing high-energy turning maneuvers and maximizing battery endurance.

B. Algorithmic Workflow in Mission Planner

1. **Boundary Definition:** The operator defines spatial vertices on the map interface (**Plan Tab** → **Draw Polygon**).
2. **Path Synthesis:** Selecting **Auto WP** → **Survey (Grid)** opens a spatial math engine that computes:

$$\text{Grid Spacing} = 2 \times \text{Altitude} \times \tan\left(\frac{\text{FOV}_{\text{horizontal}}}{2}\right) \times (1 - \text{SideLap})$$

3. **Waypoint Mapping:** Waypoints are sequentially populated with embedded commands (e.g., MAV_CMD_NAV_WAYPOINT, speed commands, and camera trigger commands like MAV_CMD_DO_SET_CAM_TRIG_DIST).

2. Hardware Interface & Mission Telemetry Specifications

A robust search system relies on dual MAVLink communication channels:

1. **Ground-to-Air Link:** Long-range telemetry link connecting the Ground Control Station (GCS) to the Flight Controller (FC).
2. **Air-to-Air Link:** High-speed local serial link connecting the Onboard Companion Computer (e.g., NVIDIA Jetson, Raspberry Pi) to the Flight Controller.

A. Communication Protocol & Parameter Setup

Parameter Interface	Port / Link Type	Protocol	Recommended Baud Rate	Primary Function
GCS Telemetry	TELEM1 (UART) / RF Radio	MAVLink v2	57,600 bps	Real-time map tracking, mission upload, manual override monitoring.
Companion Computer	TELEM2 / SERIAL2 (UART)	MAVLink v2	921,600 bps	High-speed telemetry, onboard vision trigger, dynamic mode override.

- **Flight Controller Configuration (ArduPilot):**
 - SERIAL2_PROTOCOL = 2 (MAVLink 2)
 - SERIAL2_BAUD = 921 (921,600 bps)
 - STREAMRATE / Data Stream Frequencies set to ≥ 10 Hz for ATTITUDE and GLOBAL_POSITION_INT.

B. Mission Upload Sequence (MAVLink Handshake)

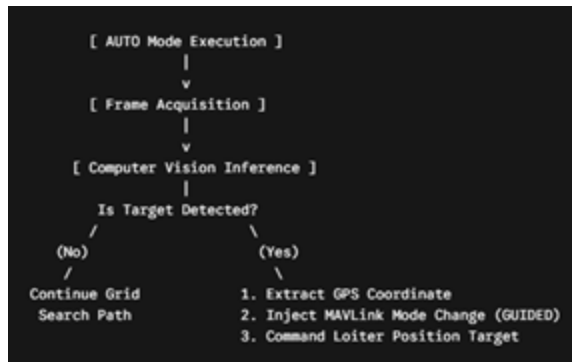
When the operator clicks **Write WPs** in Mission Planner, the GCS and FC execute a strict transactional handshake:

1. **GCS → FC:** Sends MISSION_COUNT containing the total number of survey waypoints.
2. **FC → GCS:** Requests individual waypoints sequentially via MISSION_REQUEST_INT (Index 0,1,2 ...N).
3. **GCS → FC:** Transmits each MISSION_ITEM_INT frame containing latitude, longitude, altitude, and frame coordinates.
4. **FC → GCS:** Validates boundary integrity and emits a MISSION_ACK (Acknowledge) packet.

3. Onboard Target Detection & Automated Mode Transition

A. Architecture of Onboard Script Integration

The companion computer processes real-time imagery from an onboard camera payload. A computer vision pipeline (e.g., OpenCV, YOLO, or thermal thresholding) continuously evaluates incoming frames during the AUTO lawnmower search phase.



B. Dynamic Transition to GUIDED Mode

Upon target detection, the companion script overrides the pre-programmed AUTO mission by issuing MAVLink mode command packets directly to the flight controller over the high-speed serial link:

1. **State Interruption:** The script interrupts the sequential AUTO navigation stack.
2. **Mode Switch Command:** Sends a MAV_CMD_DO_SET_MODE packet setting the custom mode to GUIDED (Mode ID 4 for ArduCopter, Mode ID 15 for ArduPlane).
3. **Position Target Injection:** Immediately follows the mode switch with a SET_POSITION_TARGET_GLOBAL_INT packet containing the calculated GPS coordinates (latitude, longitude, altitude) of the detected target.
4. **Hover / Orbit Execution:** The drone halts its grid progression and enters a loiter/orbit maneuver around the target coordinates.

//can be included if needed:

Automate Mode Switch to GUIDED on Target Detection: Companion Computer / MAVLink Script.

1. Standard ArduPilot mission plans run sequentially in AUTO mode; switching to GUIDED dynamically upon detecting a search condition requires an onboard companion computer (e.g., Raspberry Pi or NVIDIA Jetson) running a vision model and a MAVLink communication script.
2. Use pymavlink or DroneKit on the companion computer to monitor target conditions. When the search condition is met, trigger a mode change to GUIDED:

Python

```
from pymavlink import mavutil
```

```
# Establish connection with flight controller (Telem2/UART)
```

```
master = mavutil.mavlink_connection('/dev/ttyAMA0', baud=57600)
```

```
master.wait_heartbeat()
```



```

# Flag set by your camera vision / search condition model
search_condition_met = True

if search_condition_met:
    # Retrieve mode mapping and send GUIDED mode change command
    mode_id = master.mode_mapping()['GUIDED']
    master.mav.set_mode_send(
        master.target_system,
        mavutil.mavlink.MAV_MODE_FLAG_CUSTOM_MODE_ENABLED,
        mode_id
    )
    # Optional: Send loiter coordinates or position target over MAVLink

```

4. Ground Control Station (GCS) Observations

When the companion computer script executes a target detection and mode switch, specific real-time telemetry updates and visual cues occur across the Mission Planner GCS interface:

A. HUD & Status Panel Changes

- **Flight Mode Box:** The Primary Flight Display (HUD) flight mode text instantly changes from **AUTO** to **GUIDED**.
- **Custom MAVLink Messages (STATUSTEXT):** Custom log messages generated by the onboard script appear in bright yellow or red across the lower center of the HUD (e.g., STATUSTEXT: Target Acquired - Diverting to Guided).

B. Map & Trajectory Interface Changes

- **Departure from Survey Line:** The vehicle icon immediately deviates from the orange active mission leg line connecting the current grid waypoints.
- **Guided Marker Creation:** A new target marker appears on the map display at the coordinates injected by the companion computer.
- **Vector Line:** An active solid line links the vehicle's current location to the target marker.
- **Flight Trace Pattern:** The historical trail line transitions from straight grid lines to a tight circle or loiter pattern centered around the target marker.

C. Quick / Telemetry Tab Data Updates

- **Active Waypoint Index (wpno):** Freezes or clears the active mission index count.

- **Target Distance (distToHome / wp_dist):** wp_dist updates to reflect the remaining straight-line vector distance between the drone's present coordinate and the newly commanded target coordinate.

5. Fail-Safe & Safety Considerations

To prevent loss of control during automated mode changes:

1. **RC Transmitter Manual Override:** The remote pilot can instantly override GUIDED mode by toggling a physical switch on the RC transmitter assigned to ALT_HOLD, LOITER, or STABILIZE. ArduPilot prioritizes pilot switch positions over serial MAVLink inputs.
2. **Companion Link Failsafe:** If the companion computer script crashes or freezes mid-search, the flight controller will continue its original AUTO grid mission unless configured to return to launch via FS_GCS_ENABLE or secondary script heartbeat monitoring.
3. **Geofence Enforcement:** ArduPilot boundary fences remain active in GUIDED mode. If a target is detected outside the pre-programmed fence boundary, the flight controller rejects the SET_POSITION_TARGET packet and stays within the fence boundary.

Combining Mission Planner's automated polygon survey tools with onboard edge computing creates an efficient search-and-intervention platform. The pre-planned grid pattern ensures full spatial coverage without operator fatigue, while onboard vision processing enables fast, sub-second responses to targets without relying on human monitoring or high-bandwidth ground video links.

Design Overview

FRAME DESIGN AND ANALYSIS

Electronic Components

Electronic Components Selection

1.1 Motor and Propeller

The total weight of the drone is 2 kg. Since a quadcopter configuration is used, the required hover thrust per motor is:

$$\text{Hover Thrust per Motor} = 2000 / 4 = 500 \text{ g} \quad (1.1)$$

For safe flight, a minimum thrust-to-weight ratio (TWR) of 2 : 1 is required. Hence, the minimum required thrust per motor is:

$$\text{Required Thrust per Motor} = 2 \times 500 = 1000 \text{ g} \quad (1.2)$$

Based on this requirement, the SURPASS HOBBY Bat S2812 1100 KV motor, which delivers a maximum thrust of 2950 g per motor on a 4S LiPo battery with a 1245 propeller, is selected.

The achieved thrust-to-weight ratio is:

$$\text{TWR} = 2950 / 500 = 5.9 \geq 2 \quad (1.3)$$

Thus, the selected propulsion system comfortably satisfies the thrust requirement for a 2.0 kg quadcopter. The 1245 propeller generates high static thrust at lower RPMs, enhancing hover efficiency and flight stability while effectively leveraging the torque capability of the S2812 motor.

1.2 ESC and Battery

Speed Controller: A Foxeer Reaper F4 65A ESC manages motor power. Given the motor's peak draw of 24.5 A, the 65A rating provides substantial thermal margin against overheating.

Battery: A 4S 4500 mAh 35C LiPo battery supplies total system power.

$$\text{Current Capability: } I_{\text{max}} = 4.5 \text{ Ah} \times 35\text{C} = 157.5 \text{ A}$$

Estimated Flight Time: Assuming an average hover current draw of 28 A and 80% usable capacity:

$$\text{Flight Time} = (4.5 \times 0.8 \times 60) / 28 \approx 7.7 \text{ min}$$

1.3 Receiver, Transmitter, and Telemetry System

FlySky FS-iA10B RC Receiver: Handles manual flight control and emergency override over 10 channels bound to a 2.4 GHz transmitter.

433MHz 100mW Telemetry Radio: Establishes a bidirectional MAVLink data link between the flight controller and Ground Control Station (Mission Planner).

Table 1.1: Radio System and Telemetry Specifications

Feature	Specification
RC Frequency & Protocol	2.405 – 2.475 GHz / AFHDS 2A (10 Channels)
RC Receiver Sensitivity	105 dBm
Telemetry Power & Link	433 MHz at 100 mW (20 dBm) / MAVLink Serial
Control & Telemetry Range	Up to 500 m

1.4 Flight Controller, Onboard Computer, and Power Regulator

Pixhawk 6C Mini Flight Controller: Runs PX4/ArduPilot firmware, integrating redundant IMUs for attitude estimation, managing PWM/DShot motor outputs, **and** executing **Return-to-Launch (RTL) safety modes**.

Raspberry Pi 5 (8GB RAM): Serves as the primary companion computer (2.4 GHz quad-core ARM Cortex-A76) to execute real-time computer vision and mission decisions.

SmartElex TPS5430 Buck Module (5V, 3A): Steps down the 4S battery voltage to a stable 5V, 3A supply to power the Raspberry Pi 5 without brownouts during high motor loads.

1.5 GNSS System

An **M10 GPS Module** provides accurate position data (latitude, longitude, altitude) to the Pixhawk 6C Mini. It supports multi-constellation reception, a 10 Hz update rate, 1.5 m accuracy, and a cold-start TTFF of 24 seconds for reliable loitering and waypoint navigation.

1.6 Camera

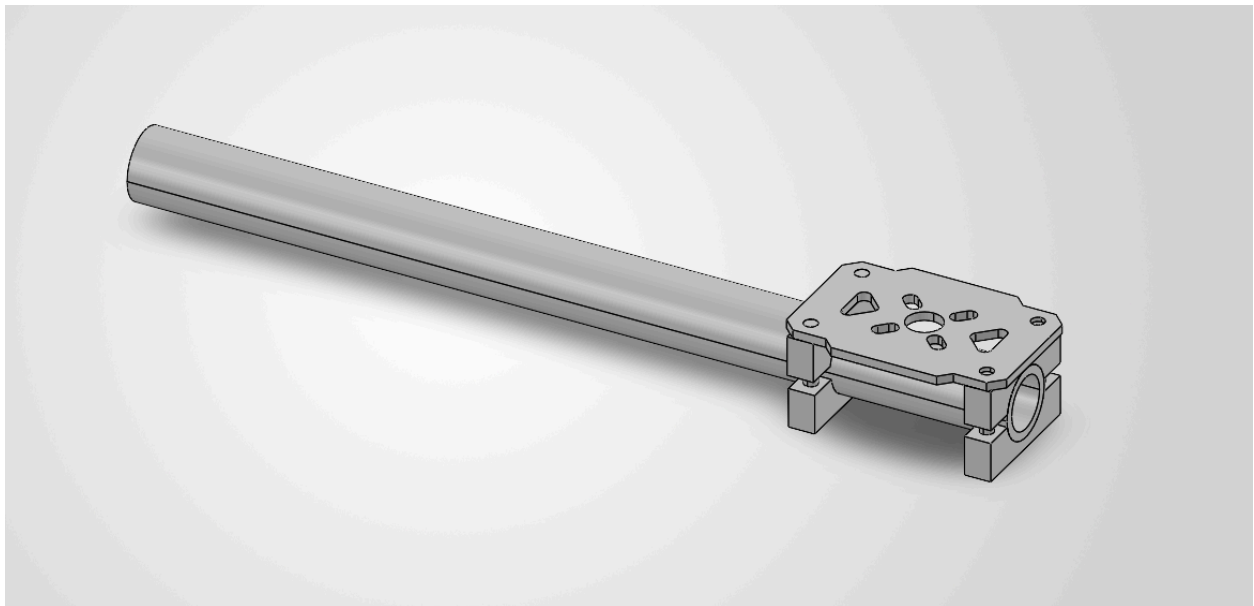
The **Arducam 1.58MP IMX296 Color Global Shutter Camera** (M12 lens) handles target detection. The global shutter exposes all pixels simultaneously, completely eliminating rolling-shutter motion blur during flight. It interfaces via MIPI CSI to the Raspberry Pi 5.

Table 1.2: Arducam IMX296 Camera Module Specifications

Feature	Specification
Sensor & Shutter	Sony IMX296 Color CMOS (Global Shutter)
Resolution & Speed	1.58 Megapixels (1456 × 1088) @ 60 fps
Interface & Mount	MIPI CSI-2 / M12 Lens Mount
Compatibility	Raspberry Pi 5 / Pi 4

Components

1. Pixhawk 6c mini
2. Telemetry radio 433MHz 100mW
3. M10 gps module
4. FlySky FS-IA10B Radio Receiver
5. Foxeer Reaper F4 65A ESC
6. SURPASS HOBBY Bat S2812 14 Pole FPV MOTOR 1100 KV
7. Battery
8. Raspberry Pi 5 Model 8GB RAM
9. SmartElex TPS5430 Buck Module - 5V, 3A
10. Arducam 1.58MP IMX296 Color Global Shutter Camera Module with M12 Lens for Raspberry Pi



Esc and battery

Receiver transmitter

Fc and on board rpi

Gnss

Camera

2 pages hopefully

Autonomous Navigation Strategy

1. System Architecture Overview

The autonomous software stack is deployed on a Raspberry Pi companion computer running Ubuntu and ROS 2, interfaced with an ArduPilot-based Flight Controller Unit (FCU) via MAVLink over a dedicated hardware UART bus (TELEM2).

TABLE 1: ROS 2 COMPUTATION & INTERFACE MATRIX					
Topic	Message Type	Publisher	Subscriber	Frequency	Description
/camera/image_raw	sensor_msgs/msg/Image	Camera Driver	VisionControlNode	~30 Hz	Downward video frames
/addclvision/target_offset	geometry_msgs/msg/Point	VisionControlNode	MissionControlNode	~20-30 Hz	Normalized error (xy) & lock status (z)
/addclvision/decoded_digits	std_msgs/msg/String	VisionControlNode	MissionControlNode, GCS	Latched/Event	2-digit numerical access code
/addclsearch/activate	std_msgs/msg/Bool	MissionControlNode	SearchNode	Event	Triggers sweep start/halt
/addclsearch/waypoint	geometry_msgs/msg/PoseStamped	SearchNode	MissionControlNode	Step/Event	Active 3D waypoint (X,Y,Z)
/addclsearch/completed	std_msgs/msg/Bool	SearchNode	MissionControlNode	Event	Boundary sweep exhausted flag
/mavros/local_position/pose	geometry_msgs/msg/PoseStamped	MAVROS	MissionControlNode, SearchNode	~30-50 Hz	3D vehicle position & orientation
/mavros/state	mavros_msgs/msg/State	MAVROS	MissionControlNode	1 Hz	Mode, armed flag, connection state
/mavros/setpoint_position/local	geometry_msgs/msg/PoseStamped	MissionControlNode	MAVROS	20 Hz	3D position setpoint commands

2. Mission Phase Breakdown

Phase 1: Pre-Flight Initialization

- Objective: Establish sensor health, system sanity, and communication links within the 2-minute setup window.
- Operations:
 1. Boot companion computer; verify hardware serial connection (/dev/ttyAMA0 or /dev/ttyUSB0) to Pixhawk TELEM2.
 2. Confirm telemetry feed inside Mission Planner: 3D GPS lock, EKF convergence, and Home point registration at the takeoff marker.
 3. ROS 2 node graph launches via systemd/bringup script. MissionControlNode enters INIT state, streaming zero-velocity/hold-position setpoints at 20 Hz to initialize the MAVROS setpoint watchdog.
 4. Ensure the drone remains disarmed until marshal authorization.

Phase 2: Autonomous Takeoff

- **Objective:** Clean vertical climb to mission altitude without lateral drift.
- **Control Mechanics:**
 - Flight mode requested: **GUIDED**.
 - Service call: `/mavros/cmd/arming` with **value: True**.
 - Position command set: (X=0.0, Y=0.0, Z=3.0)
 - State transition condition: Ascent altitude threshold check
 - Once the vehicle stabilizes at 3 meters, the FSM transitions to **TRANSIT_TO_RECON**

Phase 3: Drone Search (Lawnmower Technique)

- **Objective:** Ensure 100% sensor coverage of the reconnaissance arena within the allotted flight envelope.

Automatic Entry Calculation: The software autonomously calculates the nearest corner of the bounding box relative to the takeoff coordinate (0, 0). It designates that point $[(X_{\min}, Y_{\min}, Z=3.0)]$ as the entry waypoint and routes straight from the 3 meter hover into the first sweep lane without any hardcoded transition point.

Coverage Geometry:

- The boustrophedon(lawnmower) sweep is computed dynamically at node startup using the declared boundary parameters.
- Sweep lines run parallel across X with step increments $\Delta X = \{\text{lane_spacing}\}$, while traversing between $Y_{\min}$ and $Y_{\max}$ to ensure continuous coverage.

State Simplification: The state machine simplifies from requiring a separate transit state to a clean transition: once takeoff reaches 3.0m, it activates `search_node` immediately. The search node's very first waypoint is the entry corner itself.

Phase 4: Target Detection & QR Scanning

- **Objective:** Stop the search sweep upon marker detection, eliminate positional error, and decode the payload.
- **Detection & Interruption:**
 - **VisionControlNode** extracts the QR polygon or target contour from `/camera/image_raw`.
 - Centroid coordinates (C_x, C_y) are extracted relative to image frame dimensions (W, H):
When target is valid (z = 1.0), **MissionControlNode** sends **data: False** to `/addc/search/activate` to kill the sweep and enters **VISUAL_TRACKING**.
- **Visual Servoing (Proportional Velocity Controller):**
 - Drone commands velocity setpoints over `/mavros/setpoint_velocity/cmd_vel`:
The vehicle holds over the cache while the QR decode library resolves the two numeric digits from the barcode.

Phase 5: Sending Decoded Digits to Ground Station

- **Objective:** Instantly deliver the intelligence cache access code to the ground crew.

------(to be done by opencv people)-----

Phase 6: Return-to-Launch (RTL) & Landing

- **Objective:** Safe, autonomous recovery to the launch pad.
- **Execution:**

1. Coarse Transit to Launch Area

- Receipt of valid decoded digits or mission-complete triggers the return routine.
- Instead of switching to native ArduPilot RTL, the drone remains in **GUIDED** mode.
- **MissionControlNode** commands local position setpoint $(X = 0.0, Y = 0.0, Z = 3.0 \text{ m})$ over `/mavros/setpoint_position/local`.
- Returning at the 3-meter mission altitude prevents collisions with field obstacles while maintaining a wide camera field of view to acquire the pad.

2. Pad Detection via OpenCV

- Once local position error relative to $(0, 0)$ drops below 1.0 m , the vehicle maintains hover altitude and engages pad detection.
- The downward camera feed (`/camera/image_raw`) is processed by `VisionControlNode`:
 - Video frames are downsampled and filtered with a Gaussian blur to reduce sensor noise.
 - Adaptive thresholding or color segmentation isolates the distinct perimeter of the launch pad.
 - Spatial moments are extracted from the largest valid contour to calculate the pad's pixel centroid (C_x, C_y) .
- Normalized centroid offsets (e_x, e_y) relative to the frame center are computed and published to `/addc/vision/target_offset`, with lock status flag $Z = 1.0$ indicating valid detection.

3. Visual Servoing & Centered Descent

- `MissionControlNode` switches state to pad alignment upon receiving a valid lock ($Z > 0.5$).
- The node runs a proportional velocity controller, driving lateral velocity setpoints over `/mavros/setpoint_velocity/cmd_vel` to null horizontal offset:
 - Lateral velocity is capped at 0.4 m/s to prevent steep bank angles that could throw the pad outside the camera frame.
- **Centering and Descent Loop:**
 - **Horizontal Alignment:** The drone holds $Z = 3.0\text{ m}$ until the pixel error settles within a predefined deadband.
 - **Vertical Descent:** Once aligned, the vehicle commands a controlled downward velocity ($v_z = -0.20\text{ m/s}$) while continuously running horizontal velocity corrections to counter wind drift and ground turbulence.

4. Touchdown & Motor Disarm

- Once altitude reaches 0.4 m to 0.5 m above the platform, visual tracking is halted to avoid ground-effect turbulence and visual wash-out.

- **MissionControlNode** makes an asynchronous service call to `/mavros/set_mode` with `custom_mode: "LAND"`.
- ArduPilot completes the final vertical touchdown, detects ground contact through motor load feedback, and triggers an automatic motor disarm.

NODES ALGOS:

Algorithm 1: Mission Control FSM (`mission_control_node`)

Initialize:

```
State <- INIT
TargetAltitude <- 3.0 m
LoopRate <- 20 Hz (50 ms)
```

Every Timer Tick:

Case State:

INIT:

```
Stream zero-offset setpoint (0, 0, TargetAltitude)
If FCU connected and 2 seconds elapsed:
  Request Mode <- "GUIDED"
  Call Arming Service (True)
  If Armed and Mode == "GUIDED":
    State <- TAKEOFF
```

TAKEOFF:

```
Stream setpoint (0, 0, TargetAltitude)
If |CurrentAltitude - TargetAltitude| < 0.25 m:
  Publish /addc/search/activate <- True
  State <- SEARCHING
```

SEARCHING:

```
Stream target pose received from /addc/search/waypoint
# Interrupted asynchronously by Vision Callback:
# If TargetOffset.Z == 1.0 (Target Detected):
#   Publish /addc/search/activate <- False
#   State <- VISUAL_TRACKING
```

VISUAL_TRACKING:

```
Compute horizontal velocities:
```

```

Vx = -Kp * TargetOffset.Y
Vy = -Kp * TargetOffset.X
Vz = 0.0
Publish velocity command to /mavros/setpoint_velocity/cmd_vel
# Interrupted asynchronously by Decoded Digits Callback:
# If 2 valid digits received:
#   State <- HANDOFF_TO_RTL
#   Publish /addc/mission/trigger_rtl <- True

```

HANDOFF_TO_RTL:

```

Halt local setpoint/velocity publishing
Idle and let dedicated RTL node control the vehicle

```

Algorithm 2: Dynamic Boustrophedon Path Generator ([search_node](#))

Initialize:

```

Read dynamic arena parameters: X_min, X_max, Y_min, Y_max
Altitude <- 3.0 m, LaneSpacing <- 2.0 m
Waypoints <- []
CurrentIndex <- 0
Active <- False

```

On Startup:

```

# Generate boustrophedon sweep starting at the nearest corner (X_min, Y_min)
x <- X_min
sweep_up <- True
While x <= X_max:
  If sweep_up:
    Append (x, Y_min, Altitude) to Waypoints
    Append (x, Y_max, Altitude) to Waypoints
  Else:
    Append (x, Y_max, Altitude) to Waypoints
    Append (x, Y_min, Altitude) to Waypoints
  x <- x + LaneSpacing
  sweep_up <- NOT sweep_up

```

Callback OnSearchActivate(msg):

```

Active <- msg.data
If Active:
  CurrentIndex <- 0
  Publish Waypoints[CurrentIndex] to /addc/search/waypoint

```

```

Callback OnDroneLocalPose(current_pose):
  If NOT Active or Waypoints is Empty:
    Return

  Target <- Waypoints[CurrentIndex]
  DistanceToTarget <- sqrt((current_pose.X - Target.X)^2 + (current_pose.Y -
Target.Y)^2)

  If DistanceToTarget < 1.0 m:
    CurrentIndex <- CurrentIndex + 1
    If CurrentIndex < Length(Waypoints):
      Publish Waypoints[CurrentIndex] to /addc/search/waypoint
    Else:
      Publish /addc/search/completed <- True
      Active <- False

```

Algorithm 3: Target Detection & QR Payload Extractor ([vision_control_node](#))

```

Initialize:
  Subscribe to /camera/image_raw (~30 Hz)
  Initialize OpenCV QR Detector

Callback OnCameraFrame(frame):
  Convert frame to OpenCV BGR format
  (Height, Width) <- Dimensions(frame)
  Center <- (Width / 2, Height / 2)

  (payload, bbox) <- DetectAndDecodeQR(frame)

  If bbox is detected:
    Centroid <- CalculateCenter(bbox)
    # Compute normalized errors in range [-1.0, 1.0]
    Error_X <- (Centroid.X - Center.X) / (Width / 2.0)
    Error_Y <- (Centroid.Y - Center.Y) / (Height / 2.0)

    # Publish tracking error with lock flag (Z = 1.0)
    Publish /addc/vision/target_offset <- Point(Error_X, Error_Y, 1.0)

  If payload contains characters:

```

```

    digits <- ExtractRegexDigits(payload) # Expecting 2 numeric characters
    If Length(digits) == 2:
        Publish /addc/vision/decoded_digits <- digits
        TransmitToGroundStation(digits) # Via UDP socket or Wi-Fi relay
    Else:
        # No target in frame
        Publish /addc/vision/target_offset <- Point(0.0, 0.0, 0.0)

```

Algorithm 4: Return-to-Launch & Recovery (rtl_node)

Initialize:

```

    Subscribe to /addc/mission/trigger_rtl
    Subscribe to /mavros/state and /mavros/local_position/pose

```

Callback OnRTLTrigger(msg):

```

    If msg.data == True:
        Call MAVROS Service: /mavros/set_mode with custom_mode <- "RTL"
        Log "RTL Commanded - Returning to Launch Coordinates"

```

Continuous Monitor Loop:

```

    If Mode == "RTL":
        If VehicleState.landed == True or CurrentAltitude < 0.15 m:
            Call MAVROS Service: /mavros/cmd/arming with value <- False (Disarm)
            Log "Mission Completed: Touchdown and Motors Disarmed"

```

Actual Codes

MISSION_CONTROL node:

```

#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped, TwistStamped, Point
from std_msgs.msg import Bool, String
from mavros_msgs.srv import CommandBool, SetMode
from mavros_msgs.msg import State

class MissionState:
    INIT = 0
    TAKEOFF = 1
    SEARCHING = 2
    VISUAL_TRACKING = 3
    HANDOFF_TO_RTL = 4

```

```

class MissionControlNode(Node):
    def __init__(self):
        super().__init__('mission_control_node')

        self.state = MissionState.INIT
        self.current_drone_state = State()
        self.current_pose = PoseStamped()
        self.target_pose = PoseStamped()
        self.decoded_code = None

        self.pos_setpoint_pub = self.create_publisher(PoseStamped, '/mavros/setpoint_position/local', 10)
        self.vel_setpoint_pub = self.create_publisher(TwistStamped, '/mavros/setpoint_velocity/cmd_vel', 10)
        self.search_activate_pub = self.create_publisher(Bool, '/addc/search/activate', 10)
        self.rtl_trigger_pub = self.create_publisher(Bool, '/addc/mission/trigger_rtl', 10)

        self.arming_client = self.create_client(CommandBool, '/mavros/cmd/arming')
        self.set_mode_client = self.create_client(SetMode, '/mavros/set_mode')

        self.state_sub = self.create_subscription(State, '/mavros/state', self.drone_state_cb, 10)
        self.local_pos_sub = self.create_subscription(PoseStamped, '/mavros/local_position/pose',
self.local_pos_cb, 10)
        self.search_wp_sub = self.create_subscription(PoseStamped, '/addc/search/waypoint',
self.search_wp_cb, 10)
        self.target_offset_sub = self.create_subscription(Point, '/addc/vision/target_offset',
self.vision_target_cb, 10)
        self.digits_sub = self.create_subscription(String, '/addc/vision/decoded_digits', self.digits_cb, 10)

        self.timer = self.create_timer(0.05, self.control_loop)
        self.loop_count = 0

    def drone_state_cb(self, msg):
        self.current_drone_state = msg

    def local_pos_cb(self, msg):
        self.current_pose = msg

    def search_wp_cb(self, msg):
        if self.state == MissionState.SEARCHING:
            self.target_pose = msg

    def digits_cb(self, msg):
        if not self.decoded_code and msg.data:
            self.decoded_code = msg.data
            self.trigger_external_rtl()

    def vision_target_cb(self, msg):
        if msg.z > 0.5 and self.state == MissionState.SEARCHING:
            stop_search = Bool()
            stop_search.data = False

```

```

        self.search_activate_pub.publish(stop_search)
        self.state = MissionState.VISUAL_TRACKING

    if self.state == MissionState.VISUAL_TRACKING:
        cmd = TwistStamped()
        cmd.header.stamp = self.get_clock().now().to_msg()
        kp = 1.0
        cmd.twist.linear.x = float(-kp * msg.y)
        cmd.twist.linear.y = float(-kp * msg.x)
        cmd.twist.linear.z = 0.0
        self.vel_setpoint_pub.publish(cmd)

    def trigger_external_rtl(self):
        self.state = MissionState.HANDOFF_TO_RTL
        trigger_msg = Bool()
        trigger_msg.data = True
        self.rtl_trigger_pub.publish(trigger_msg)

    def control_loop(self):
        self.loop_count += 1

        if self.state == MissionState.INIT:
            self.target_pose.pose.position.x = 0.0
            self.target_pose.pose.position.y = 0.0
            self.target_pose.pose.position.z = 3.0
            self.pos_setpoint_pub.publish(self.target_pose)

            if self.loop_count > 40 and self.current_drone_state.connected:
                if self.current_drone_state.mode != "GUIDED":
                    mode_req = SetMode.Request(custom_mode="GUIDED")
                    self.set_mode_client.call_async(mode_req)
                elif not self.current_drone_state.armed:
                    arm_req = CommandBool.Request(value=True)
                    self.arming_client.call_async(arm_req)
                else:
                    self.state = MissionState.TAKEOFF

            elif self.state == MissionState.TAKEOFF:
                self.pos_setpoint_pub.publish(self.target_pose)
                if abs(self.current_pose.pose.position.z - 3.0) < 0.25:
                    self.state = MissionState.SEARCHING
                    act_msg = Bool()
                    act_msg.data = True
                    self.search_activate_pub.publish(act_msg)

            elif self.state == MissionState.SEARCHING:
                self.pos_setpoint_pub.publish(self.target_pose)

            elif self.state == MissionState.HANDOFF_TO_RTL:

```

```

        pass

def main(args=None):
    rclpy.init(args=args)
    node = MissionControlNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

SEARCH_CONTROL node:

```

#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
from std_msgs.msg import Bool

class SearchNode(Node):
    def __init__(self):
        super().__init__('search_node')

        self.declare_parameter('recon_x_min', 8.0)
        self.declare_parameter('recon_x_max', 20.0)
        self.declare_parameter('recon_y_min', -6.0)
        self.declare_parameter('recon_y_max', 6.0)
        self.declare_parameter('search_altitude', 3.0)
        self.declare_parameter('lane_spacing', 2.0)

        self.target_waypoint_pub = self.create_publisher(PoseStamped, '/addc/search/waypoint', 10)
        self.search_completed_pub = self.create_publisher(Bool, '/addc/search/completed', 10)

        self.activate_sub = self.create_subscription(
            Bool, '/addc/search/activate', self.activate_callback, 10
        )
        self.local_pos_sub = self.create_subscription(
            PoseStamped, '/mavros/local_position/pose', self.pos_callback, 10
        )

        self.waypoints = []
        self.current_idx = 0
        self.active = False
        self.current_pos = None

        self.generate_search_grid()

    def generate_search_grid(self):

```

```

x_min = self.get_parameter('recon_x_min').value
x_max = self.get_parameter('recon_x_max').value
y_min = self.get_parameter('recon_y_min').value
y_max = self.get_parameter('recon_y_max').value
alt = self.get_parameter('search_altitude').value
spacing = self.get_parameter('lane_spacing').value

self.waypoints = []
x = x_min
sweep_up = True

while x <= x_max:
    if sweep_up:
        self.waypoints.append((x, y_min, alt))
        self.waypoints.append((x, y_max, alt))
    else:
        self.waypoints.append((x, y_max, alt))
        self.waypoints.append((x, y_min, alt))
    x += spacing
    sweep_up = not sweep_up

def activate_callback(self, msg):
    self.active = msg.data
    if self.active and self.waypoints:
        self.current_idx = 0
        self.publish_current_waypoint()

def pos_callback(self, msg):
    if not self.active or not self.waypoints or self.current_idx >= len(self.waypoints):
        return

    self.current_pos = msg.pose.position
    target_x, target_y, _ = self.waypoints[self.current_idx]
    dx = self.current_pos.x - target_x
    dy = self.current_pos.y - target_y
    dist = (dx**2 + dy**2)**0.5

    if dist < 1.0:
        self.current_idx += 1
        if self.current_idx < len(self.waypoints):
            self.publish_current_waypoint()
        else:
            done = Bool()
            done.data = True
            self.search_completed_pub.publish(done)
            self.active = False

def publish_current_waypoint(self):
    x, y, z = self.waypoints[self.current_idx]

```



```

wp = PoseStamped()
wp.header.stamp = self.get_clock().now().to_msg()
wp.header.frame_id = "map"
wp.pose.position.x = float(x)
wp.pose.position.y = float(y)
wp.pose.position.z = float(z)
self.target_waypoint_pub.publish(wp)

```

```

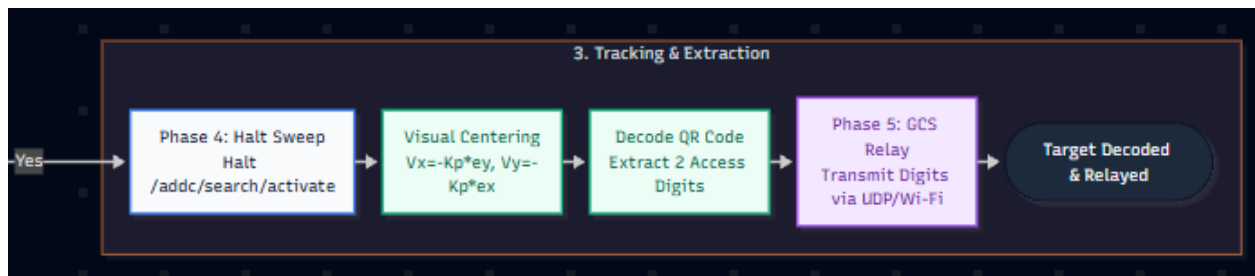
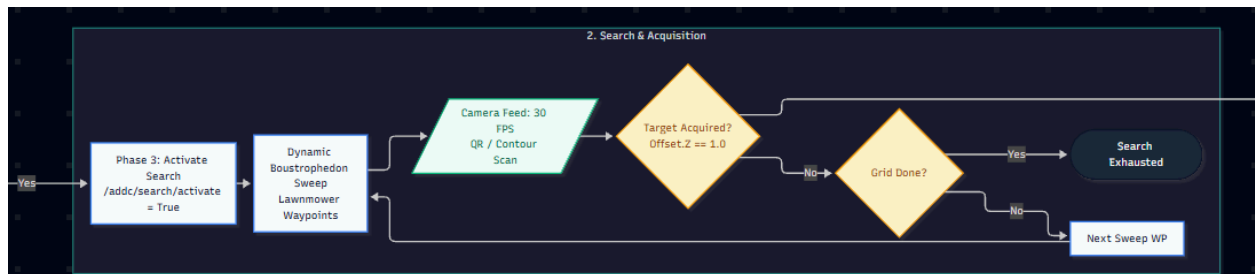
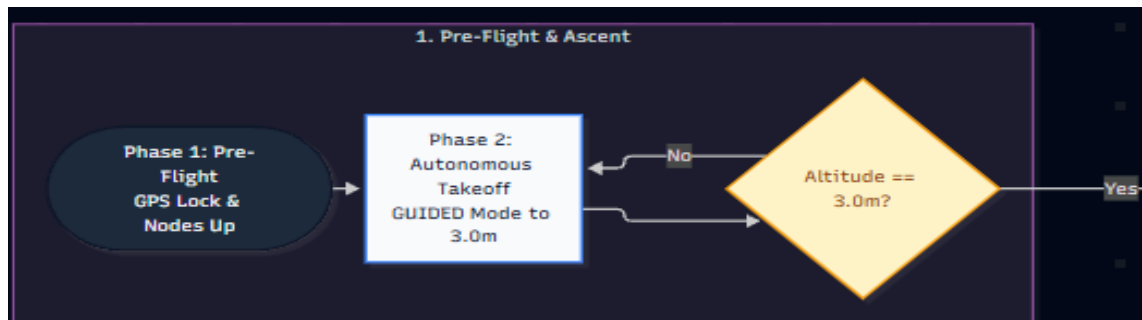
def main(args=None):
    rclpy.init(args=args)
    node = SearchNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

```

```

if __name__ == '__main__':
    main()

```



Different Search algos along with their advantages and disadvantages:

1. **Spiral Search:**

The drone starts at a center point and flies in an expanding circle or square.

- **Advantages:**
 - **Smooth Flight:** Unlike a grid, a circular spiral allows the drone to maintain constant velocity without frequent "stop-and-turn" maneuvers, reducing battery drain and camera jitter.
 - **Center-Weighted Efficiency:** In many competitions, targets are placed toward the center of the arena. This algorithm finds those targets almost immediately.
 - **Continuous Data:** The camera always has a "fresh" view of unsearched territory without the overlap gaps often found at the ends of grid lines.
- **Disadvantages:**
 - **Poor Corner Coverage:** It is very difficult to fit a spiral into a "randomly shaped polygon." You will likely leave the corners of the arena unsearched.
 - **Mathematical Complexity:** Maintaining a perfect "cross-track" distance (the gap between the rings of the spiral) requires more complex coding than a simple grid.
 - **Varying Resolution:** As the spiral gets larger, the drone's angular velocity changes, which can affect the consistency of the image processing.

2. **Zoom Search (Hierarchical/Two-Tiered):**

High-altitude fast flight to find the "box," followed by a low-altitude descent to scan the "QR code."

- **Advantages:**
 - **Maximum Speed:** You can cover the entire 500m² arena in 60 seconds at high altitude because the camera's Field of View (FOV) is much wider.
 - **High Decode Success:** By "zooming in" (descending) once the target is found, you ensure the QR code fills the frame, making it much easier to decode than from 10 meters up.
 - **Battery Efficient:** Spends less time in the air overall by avoiding low-altitude, high-drag flight over empty areas.
- **Disadvantages:**

- Coding Complexity: Requires a "State Machine" (e.g., using MAVLink or Python) to switch the drone from "Auto Mission" to "Precision Land/Descend" and back again.
- Detection Risk: If your high-altitude "box detector" fails (due to small size or camouflage), the drone will finish the mission without ever seeing the QR code.
- Time Penalty for Altitude Change: Ascending and descending takes time. If you have multiple "false positives," you will run out of your 5-minute window.

3. **Saliency-Based Search**:

Using Computer Vision to identify "anomalies" (objects that don't look like grass/dirt) and flying toward them immediately.

- Advantages:
 - "Intelligent" Behavior: It mimics how a human pilot looks for things. It ignores 90% of the "boring" terrain and focuses only on man-made objects.
 - Fastest Discovery: If the "Intelligence Cache" is high-contrast (e.g., a white box on green grass), the drone will likely head straight for it within the first 10 seconds.
 - Adaptability: Works well in "randomly shaped polygons" because it doesn't care about the boundary; it only cares about what it sees.
- Disadvantages:
 - High Computational Load: Requires a powerful companion computer (like an NVIDIA Jetson or Raspberry Pi 4/5) to run saliency algorithms in real-time.
 - Easily Distracted: The drone might "chase" a sunlight reflection, a bright rock, or even the Ground Operative, wasting precious time on "false targets."
 - Unpredictable Path: Because the path is reactive, it's hard for the team to know exactly where the drone is going, which can be stressful during a timed competition.

4. **Lawnmower Search (Standard Grid)**:

The drone flies back and forth in parallel lines until the entire polygon is covered.

- Advantages:
 - 100% Coverage Guaranteed: If the QR code is in the polygon, a properly planned grid will fly over it. There is zero "guesswork."
 - Easiest to Program: This is a native feature in Mission Planner (Survey Grid). You don't need to write a single line of custom code.
 - Predictable Timing: You can calculate exactly how many seconds the grid will take before you even take off.
- Disadvantages:
 - Inefficient: If the target is at the very end of your grid, you waste 4 minutes flying over empty ground.
 - "Stop-and-Turn" Waste: Every time the drone reaches the end of a line, it must slow down, stop, turn, and accelerate. This is the slowest way to fly.
 - Motion Blur: The camera often shakes during the "turns" at the end of the lines, which can lead to missed detections if the target is near the boundar

Autonomous return & Landing system

Autonomous Return & Precision Landing System: System Architecture Overview

This document details the telemetry tracking, return transit, inter-node ROS communication, and closed-loop visual descent mechanics triggered once the target is decoded.

The original plan relied on injecting a blind `MAV_CMD_NAV_LAND` waypoint to execute an automatic touchdown via GPS. However, consumer GPS horizontal drift (typically 1.5 m to 3.0 m) makes direct GPS landing unviable for an elevated launch pad, where a lateral error of even 0.5 meters will cause the drone to clip the edge and roll over.

To prevent this, the recovery pipeline is split into two phases: **Coarse Transit** back to the launch coordinates at a safe cruise altitude, followed by **Vision-Guided Precision Landing** using OpenCV to center the vehicle directly over the pad before touchdown.

1. Telemetry Latching at Ground Origin

To eliminate hardcoded coordinate dependencies, the system registers origin coordinates directly from the flight controller during pre-flight initialization:

- **Global GPS Telemetry:** Subscribes to `/mavros/global_position/global` (`sensor_msgs/NavSatFix`). The first valid packet with a verified 3D fix is latched in memory as `self.home_gps = (latitude, longitude, altitude)`.
- **Local Odometry Telemetry:** Subscribes to `/mavros/local_position/pose` (`geometry_msgs/PoseStamped`). The initial Cartesian origin is locked as `self.home_local_pose = (0.0, 0.0, 0.0)`.
- **QoS Profiles:** Telemetry subscribers use a Best-Effort, depth-1 QoS policy to enforce instantaneous state reads and drop stale sensor queues.

2. Scanned Data Broadcast Pipeline

Before overriding the flight mode for return transit, target identification is transmitted across the ROS network:

- **Topic Interface:** Target intelligence is published to `/addc/vision/decoded_digits` as a `std_msgs/String`.

- **Execution:** Once the visual tracking loop confirms the decoded code, `MissionControlNode` publishes the string to alert the Ground Control Station (GCS).
- **Transition Trigger:** Publishing the code immediately triggers the `trigger_rtl()` routine inside `MissionControlNode`, shifting the internal state machine from search/tracking to return mode.

3. Coarse Transit to Launch Vicinity (Stage 1)

Instead of passing control to ArduPilot's native `RTL` or uploading a blind MAVLink landing sequence, the companion computer retains active trajectory control in `GUIDED` mode:

- **Transit Command:** `MissionControlNode` commands a local position setpoint to local origin coordinates $(X = 0.0, Y = 0.0, Z = 3.0 \text{ m})$ over `/mavros/setpoint_position/local`.
- **Altitude Holding:** Maintaining a fixed altitude of 3.0 meters ensures clean separation from ground obstacles while preserving an optimal camera Field of View (FOV) to acquire the platform upon arrival.
- **Arrival Boundary:** The node continuously calculates horizontal Euclidean separation to origin:

$$d = \sqrt{(X_{\text{current}} - X_0)^2 + (Y_{\text{current}} - Y_0)^2}$$
 When $d < 1.0 \text{ meter}$, the coarse transit is marked complete, and the system transitions from position-based waypoint navigation to optical target acquisition.

4. OpenCV Pad Acquisition & Visual Servoing (Stage 2)

Once within the launch pad footprint, lateral guidance is handed over to the onboard vision pipeline:

- **Pad Centroid Detection:**
 - Downward video frames from `/camera/image_raw` are processed by `VisionControlNode` at ~30 Hz.
 - Incoming frames pass through a Gaussian blur and adaptive thresholding to segment the geometric boundary of the elevated platform.
 - Spatial moments calculate the center pixel coordinates of the pad (C_x, C_y) .

- Normalized offsets (e_x, e_y) relative to the frame center $(W/2, H/2)$ are published along with a target lock flag ($Z = 1.0$) to `/addc/vision/target_offset`.
- **Search Fallback:** If the drone arrives at $(0, 0)$ but the pad is not within the camera frame ($Z = 0.0$), `MissionControlNode` executes a localized expanding spiral search (capped at a 2.0-meter radius) until visual lock is established.
- **Closed-Loop Alignment:**
 - `MissionControlNode` runs a proportional velocity controller that streams body-frame velocity setpoints to `/mavros/setpoint_velocity/cmd_vel`:

$$v_x = -K_p \cdot e_y$$

$$v_y = -K_p \cdot e_x$$
 - Lateral velocity is saturated at 0.4 m/s to prevent aggressive roll/pitch angles that could throw the landing pad outside the camera frame.

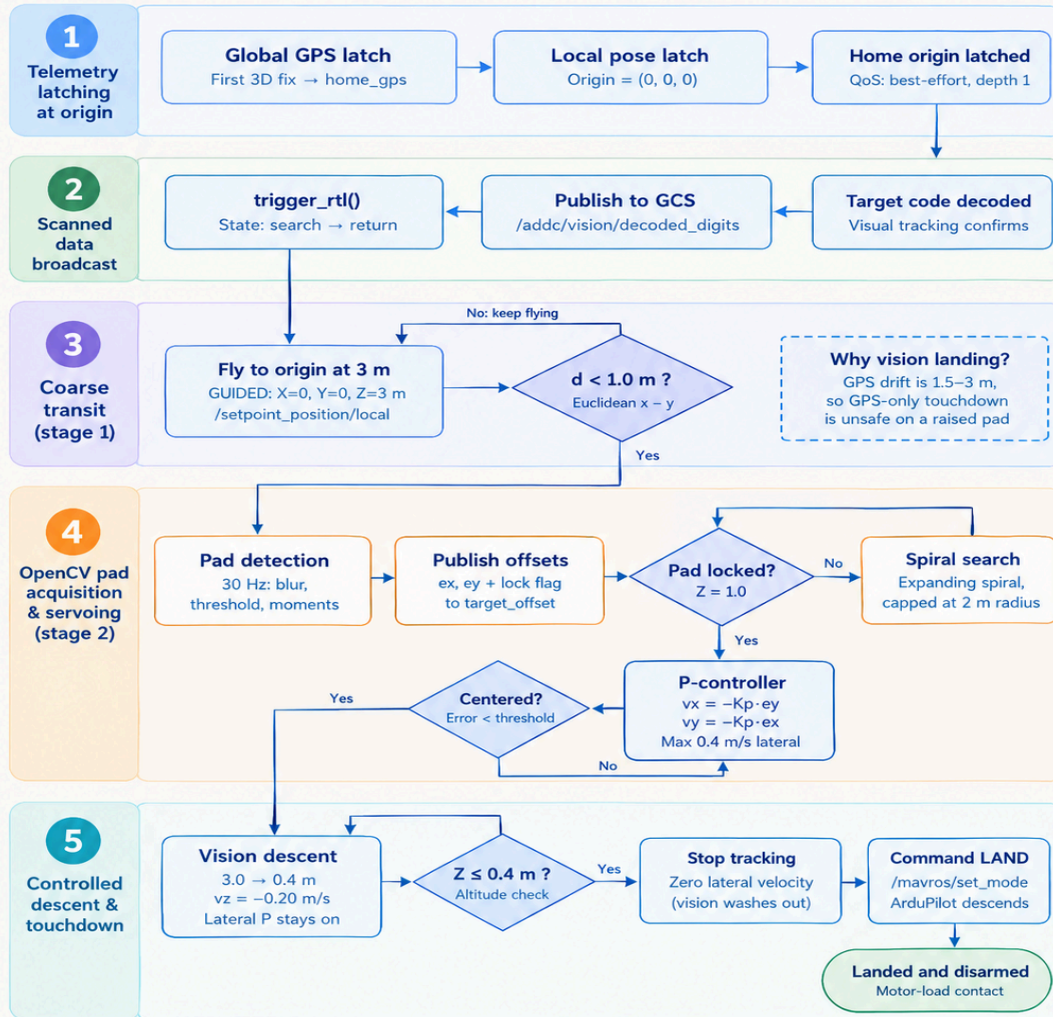
5. Controlled Descent & Touchdown Verification

Landing is executed in two stages to prevent airframe oscillation and ground-effect drift:

- **Continuous Vision-Guided Descent (3.0 m to 0.4 m):**
 - Once the horizontal pixel error falls within the centering threshold, the drone initiates vertical descent at a constant rate of $v_z = -0.20 \text{ m/s}$.
 - The lateral proportional controller remains active throughout the descent to correct for wind shear and ground-effect turbulence.
- **Terminal Touchdown ($Z \leq 0.4 \text{ m}$):**
 - Below 0.4 meters, ground proximity causes visual wash-out and propeller downwash distortion.
 - Optical tracking is terminated, horizontal velocities are zeroed, and `MissionControlNode` calls `/mavros/set_mode` with `custom_mode: "LAND"`.

Autonomous return & precision landing system

Latch home > Broadcast target > Coarse transit > OpenCV servoing > Controlled descent

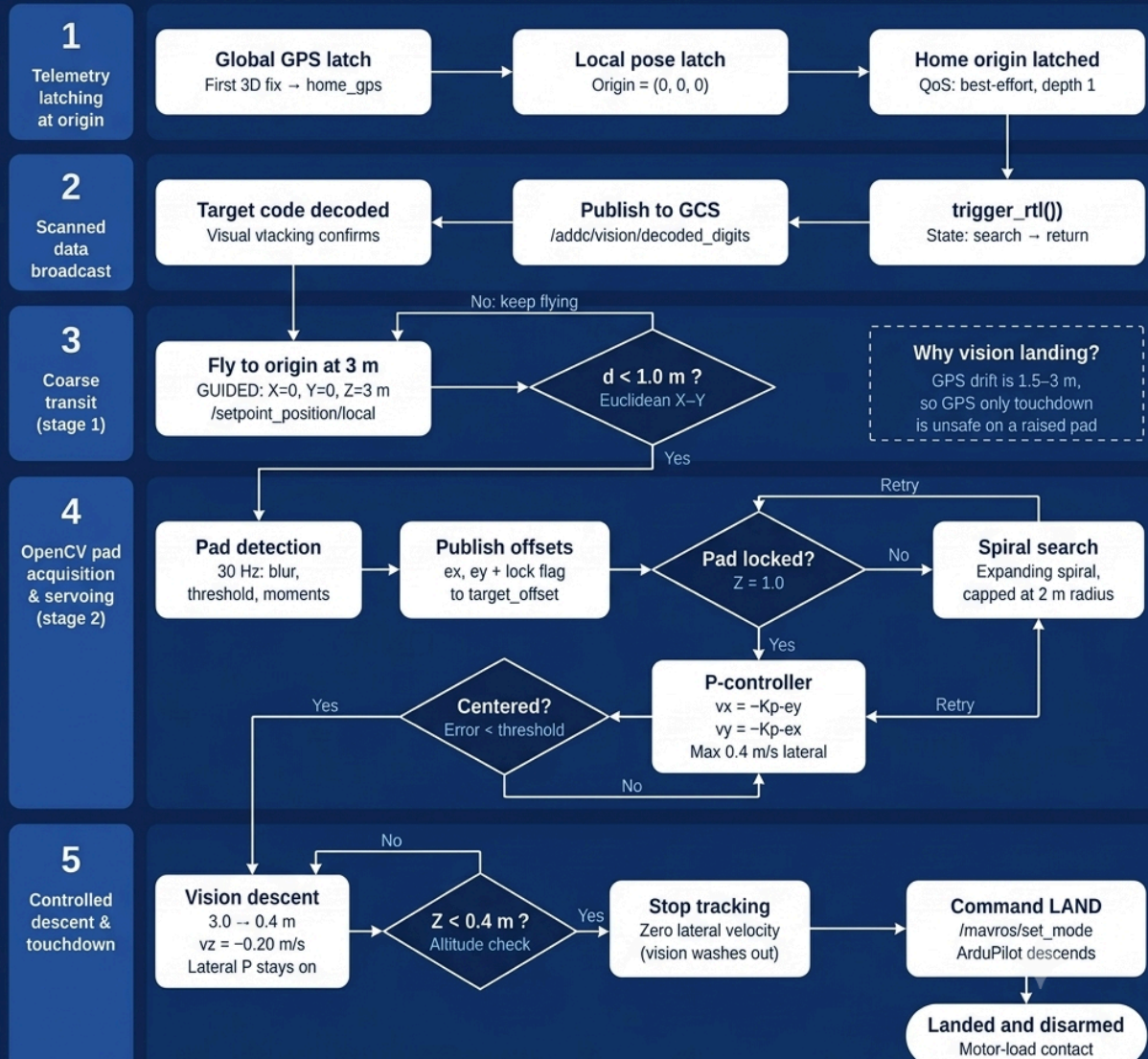


□ Action ◇ Decision ▭ Note → Flow

Autonomous return & precision landing system

Latch home > broadcast target > coarse transit > OpenCV servoing > controlled descent

□ Action ◇ Decision ▭ Note → Flow



CV and QR detection strategy

Onboard Motion QR Subsystem

Subsystem Objective: Autonomous identification and validation of two-digit numeric waypoint codes in transit, utilizing decoupled detection and sequential image enhancement on an embedded target.

1. SUBSYSTEM SPECIFICATIONS

COMPUTE CORE	Raspberry	Pi 5 8GB	SOFTWARE STACK	VALIDATION RULE
		CAPTURE STREAM	Picamera2, OpenCV, NumPy	\d.*?\d (Strict)
		960×540 BGR @ 30 FPS		

2. AS-BUILT ARCHITECTURE & PIPELINE EXECUTION

Stage I: Frame Capture & Single-Pass Detection

Synchronously captures 960×540 BGR frames via `camera.capture_array()` and converts them to grayscale. Executes `qr.detect(gray)` exactly once per iteration to locate bounding points. If corner extraction fails, the cycle exits immediately without triggering decoders.

Stage III: Four-Stage Cascaded Decode Mechanism

Stage II: Target Reticle

Overlays a central target reticle (spanning 50% frame dimensions: 25-75% of width and height). Renders orange during active scanning and shifts green on match. Bounding polygon is outlined in green immediately when corner points are found, even if payload decode fails

Corner coordinates from detection are reused directly across four sequential image variations. Decoding halts upon the first variant that matches `^[0-9]{2}$`:

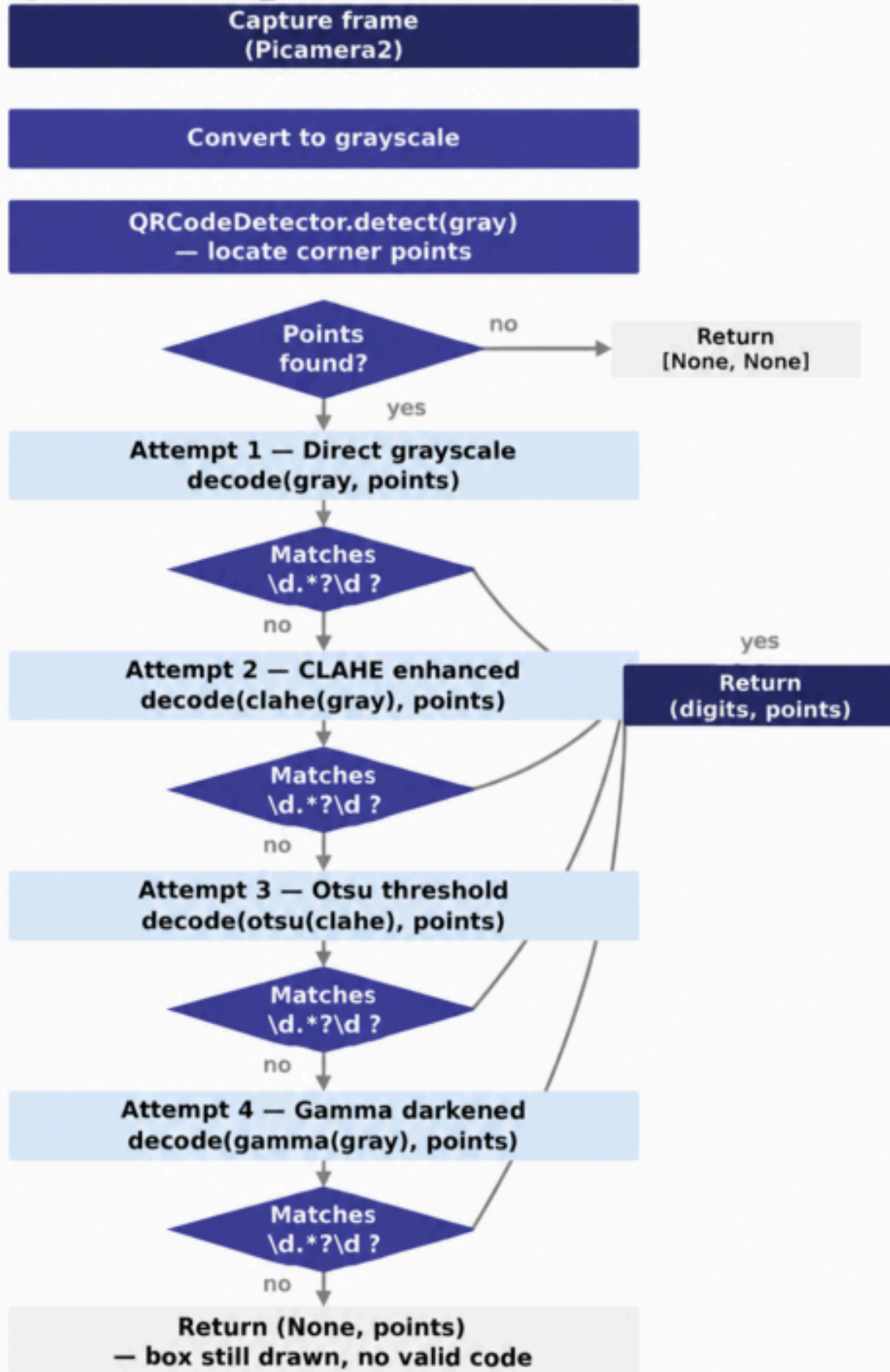
- Attempt 1 (Direct Grayscale): Raw grayscale feed evaluated directly with cached corner points.
- Attempt 2 (CLAHE Enhanced): Contrast-limited adaptive histogram equalization (`clipLimit=2.0` , `8×8` grid) for low-light or distance contrast attenuation.
- Attempt 3 (Otsu Threshold): Automatic global binarization applied over the CLAHE-enhanced image to resolve high non-binary contrast.
- Attempt 4 (Gamma LUT Darkening): Pre-computed lookup table (gamma taken approximately 3.33) applied to the raw grayscale image as a fallback for severe overexposure and surface glare.

3. IMPLEMENTATION LIMITATIONS & BOTTLENECKS

FAILURE VECTOR	CODE IMPACT	TECHNICAL ROOT CAUSE
Motion Blur Blindness	High Drop Rate	Zero edge sharpness checks (e.g., Laplacian variance) or deblur mechanisms; pixel enhancements cannot recover lost geometric edges in motion.
Static Cascade Order	Latency Penalty	Sequence runs in a fixed order regardless of lighting; under severe glare, three stages run and fail prior to reaching the gamma LUT fallback.
Unbounded Search Field	CPU Overhead	<code>qr.detect()</code> searches the full 960×540 frame every cycle, ignoring both the central HUD target reticle and temporal locality of prior hits.
Synchronous Frame Loop	Pipeline Jitter	Capture, enhancement passes, decode calls, HUD drawing, and disk writes execute sequentially on a single thread with no producer/consumer queue.

Auto-Exposure Hunting	Variable Blur	Camera controls lack fixed exposure/gain constraints, permitting the camera driver to lengthen shutter times and induce motion blur.
Vestigial Offsets	Dead Logic	offset_x and offset_y remain hardcoded to 0 throughout scan_frame() , executing as a functional no-op.

Figure 2 — scan_frame() Decision Logic



Algorithm

The algorithm implements single-pass QR point detection followed by a four-stage sequential decode cascade. The requested validation rule is `\d.*?\d`.

Algorithm

1. Capture one BGR frame using `camera.capture_array()`.
2. Convert the captured frame to grayscale.
3. Run `QRCodeDetector.detect(gray)` once to obtain QR corner points.
4. If no corner points are found, return `(None, None)` immediately.
5. Draw the detected QR bounding polygon and maintain the central target reticle for the HUD. **6.** Decode sequentially using the same detected corner points:
 - Attempt 1: decode the raw grayscale image.
 - Attempt 2: apply CLAHE enhancement, then decode.
 - Attempt 3: apply Otsu thresholding to the CLAHE image, then decode.
 - Attempt 4: apply gamma darkening to the raw grayscale image, then decode.
7. After each decode, validate the returned payload against `\d.*?\d`.
8. If the payload matches, return `(digits, points)` and stop the cascade.
9. If all four attempts fail validation, return `(None, points)`; the detected box remains available for display.

Pseudocode

```
FUNCTION scan_frame():  
    frame ← camera.capture_array()  
    gray ← grayscale(frame)  
    points ← qr.detect(gray)  
  
    IF points is None:  
        RETURN (None, None)  
  
    draw_reticle_and_box(points)  
  
    attempts ← [gray,  
                clahe(gray),  
                otsu(clahe(gray)),  
                gamma_dark(gray)]  
  
    FOR image IN attempts:  
        digits ← decode(image, points)  
        IF digits matches \d.*?\d:  
            RETURN (digits, points)  
  
    RETURN (None, points)
```

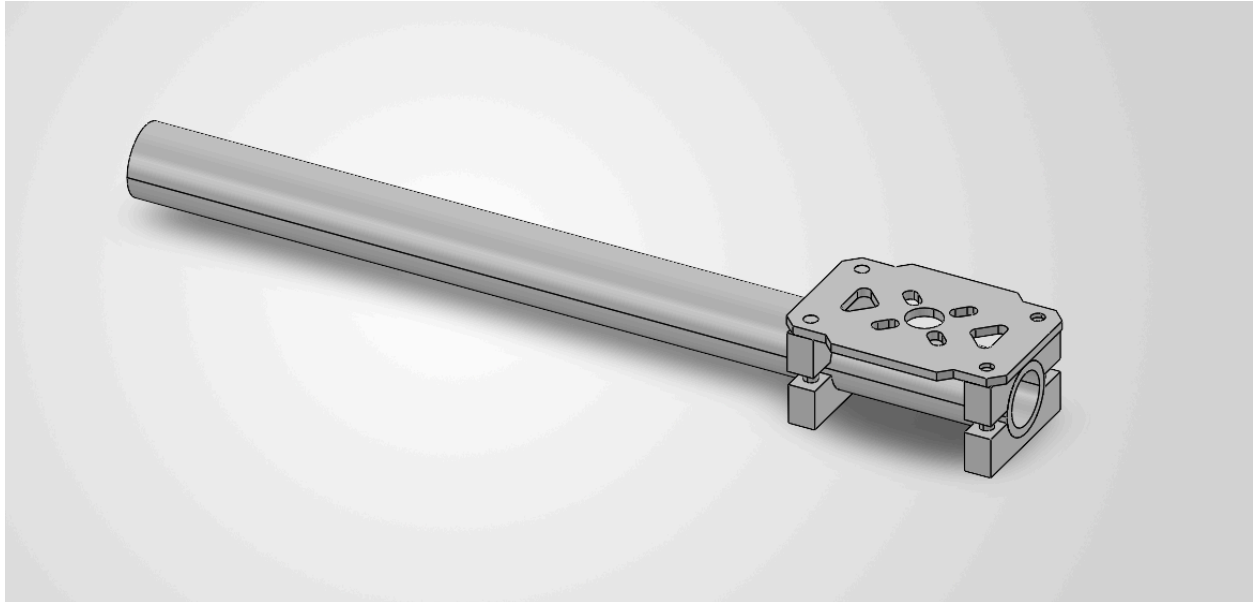
Validation note

The validation expression `\d.*?\d` requires a digit, followed by zero or more characters, followed by another digit. Decode order: Direct Grayscale → CLAHE → Otsu → Gamma Darkening. The cascade terminates immediately after the first valid match.

Any other analysis

Safety and Risk analysis

Basically all types of failsafes and prop guards



Unique Tech Development

AGENT AWARE NAVIGATION (BY PR)



Conclusion and Appendix

References

2D Drawing

Tab 14

Tab 16

Return-to-Launch (RTL) & Landing

- **Objective:** Safe, autonomous recovery to the launch pad.
- **Execution:**

1. Coarse Transit to Launch Area

- Receipt of valid decoded digits or mission-complete triggers the return routine.
- Instead of switching to native ArduPilot RTL, the drone remains in **GUIDED** mode.
- **MissionControlNode** commands local position setpoint $(X = 0.0, Y = 0.0, Z = 3.0\text{ m})$ over `/mavros/setpoint_position/local`.
- Returning at the 3-meter mission altitude prevents collisions with field obstacles while maintaining a wide camera field of view to acquire the pad.

2. Pad Detection via OpenCV

- Once local position error relative to $(0, 0)$ drops below 1.0 m , the vehicle maintains hover altitude and engages pad detection.
- The downward camera feed (`/camera/image_raw`) is processed by **VisionControlNode**:
 - Video frames are downsampled and filtered with a Gaussian blur to reduce sensor noise.
 - Adaptive thresholding or color segmentation isolates the distinct perimeter of the launch pad.
 - Spatial moments are extracted from the largest valid contour to calculate the pad's pixel centroid (C_x, C_y) .
- Normalized centroid offsets (e_x, e_y) relative to the frame center are computed and published to `/addc/vision/target_offset`, with lock status flag $Z = 1.0$ indicating valid detection.

3. Visual Servoing & Centered Descent

- **MissionControlNode** switches state to pad alignment upon receiving a valid lock ($Z > 0.5$).
- The node runs a proportional velocity controller, driving lateral velocity setpoints over `/mavros/setpoint_velocity/cmd_vel` to null horizontal offset:
 - Lateral velocity is capped at 0.4 m/s to prevent steep bank angles that could throw the pad outside the camera frame.

- **Centering and Descent Loop:**

- **Horizontal Alignment:** The drone holds $Z = 3.0 \text{ m}$ until the pixel error settles within a predefined deadband.
- **Vertical Descent:** Once aligned, the vehicle commands a controlled downward velocity ($v_z = -0.20 \text{ m/s}$) while continuously running horizontal velocity corrections to counter wind drift and ground turbulence.

4. Touchdown & Motor Disarm

- Once altitude reaches 0.4 m to 0.5 m above the platform, visual tracking is halted to avoid ground-effect turbulence and visual wash-out.
- **MissionControlNode** makes an asynchronous service call to `/mavros/set_mode` with `custom_mode: "LAND"`.
- ArduPilot completes the final vertical touchdown, detects ground contact through motor load feedback, and triggers an automatic motor disarm.